Understanding tabular foundation models: the architecture behind TabPFN

Mohit Saharan, P5, 20260417

Reference: “The architecture behind TabPFN: How transformers learn from entire tables”, C. Molnar.

Background

- In the first post (P3) about tabular foundation models (TFMs), I collected the vocabulary about their background and their implementation in code. In the second post (P4), I spent time reading through through the prior-data fitted networks (PFNs) to get a sense of how these models make predictions on unknown data without training on it.
- Before starting to write this post, I skimmed P3 to see if I felt comfortable playing with the code. I didn't feel comfortable. I felt the desire to understand the TFMs a little better to develop an appreciation for their capabilities and intuition for what I could do with them. Also, in P3, I didn't talk about how TFMs work at all because, on that day, I was overwhelmed by all the details. After yesterday's post, I got a step deeper, and today I felt like going one step further in understanding how TFMs work.
- In this post, I read through the third post by C Molnar, which talks about the architecture of TabPFN (v2) and made notes of relevant content. Actually, due to the complexity of TabPFN, his post talks about nanoTabPFN, which is a lightweight and educational reimplementation of TabPFN.

Classical ML and Tabular Foundation Models

- In classical ML, where we use training data to train the model and test data to make predictions, we may think of a row as a fundamental unit.
- TabPFN and other tabular foundation models (TFMs) do in-context learning, so at prediction time, they take in the entire dataset, combining training and testing data.
- At the same time, the fundamental unit on which TabPFN operates is individual cells of the table, or rather, their embedded representations.
- This post attempts to explain how to think in terms of cells while working with TFMs.

Table cells become computational units

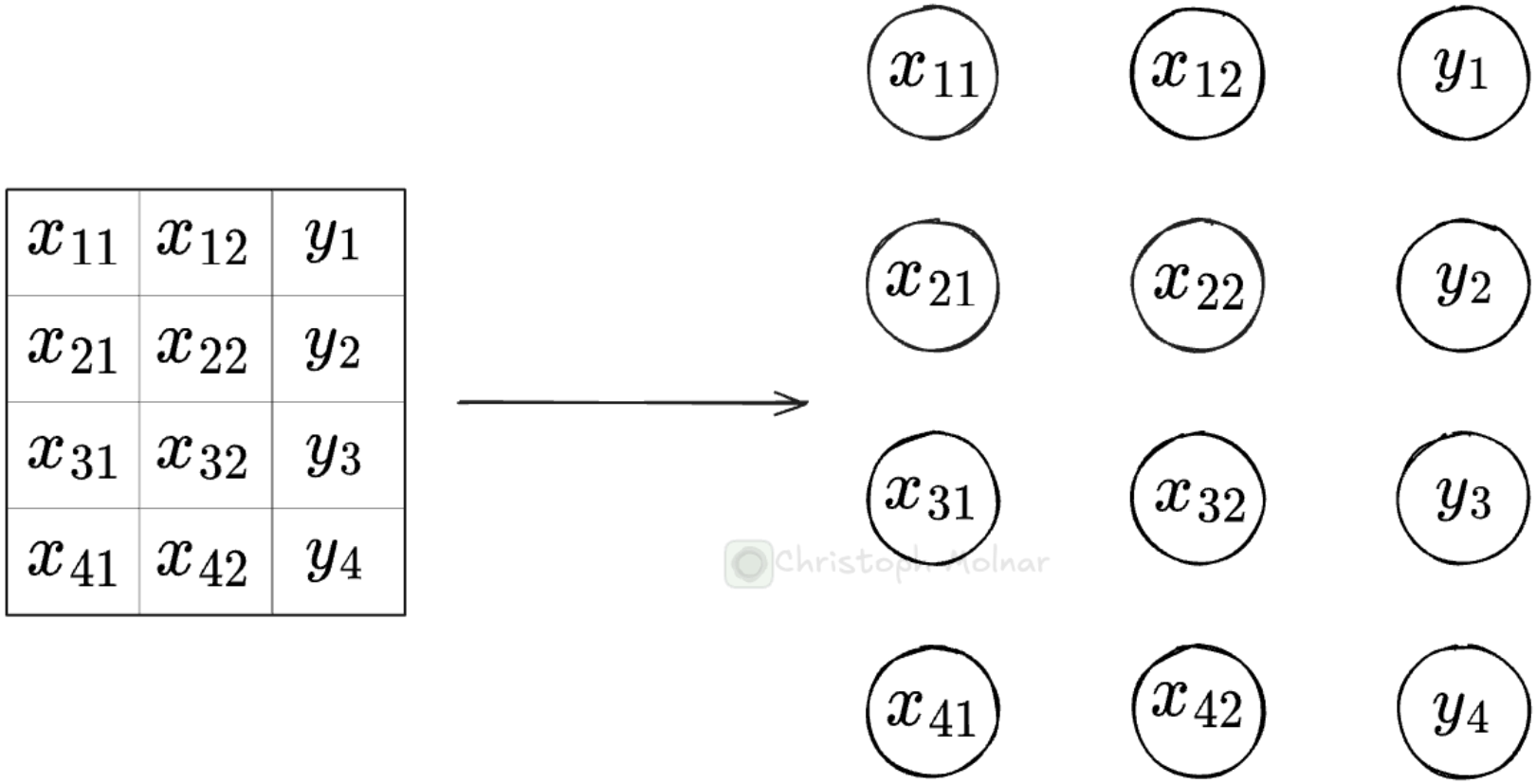
TabPFN and other TFMs use the transformer

Designed for token-based sequential data (text)

- In LLMs, the tokenized text gets embedded (each token represented by a vector).
- Transformer based architectures operate on these vectorized representations of these tokens.
- The attention mechanism acts as a way of querying the input tokens for predicting the output tokens.

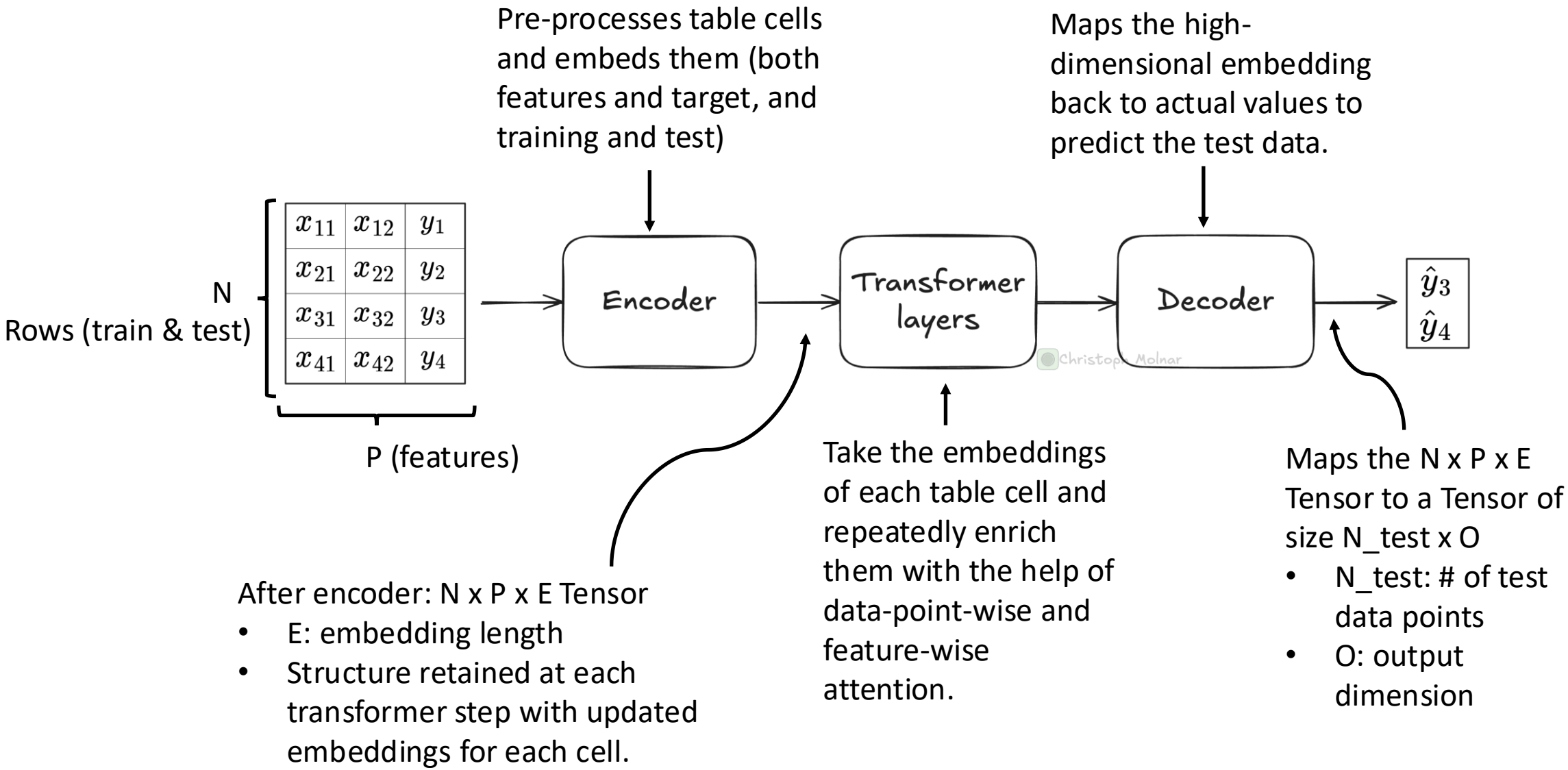
Table cells become computational units

- Tables contain scalars, not tokens → need to rethink tables to apply transformer to tables.
- For TabPFN, these fundamental units are the table cells (feature=value or target=value).

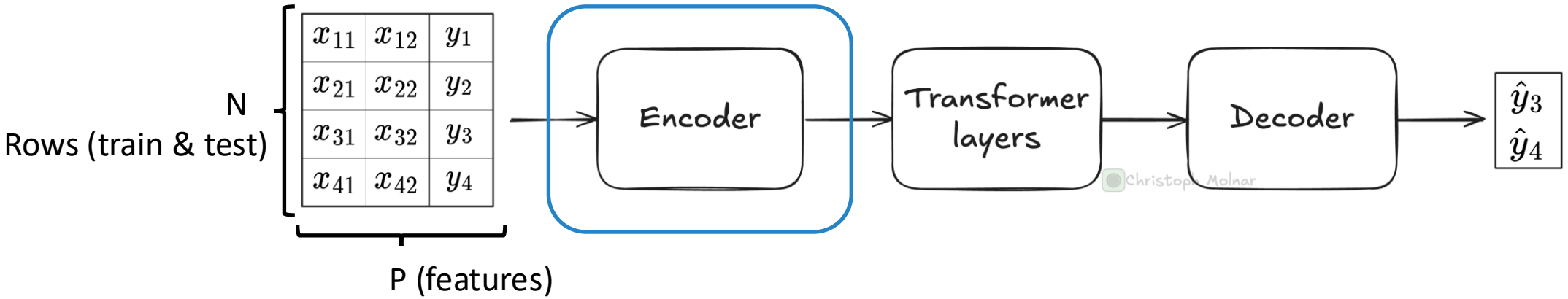


TabPFN is an encoder-decoder transformer

TabPFN and many other TFMs follow an encoder-decoder transformer architecture with this high-level structure.



The encoder pre-processes and embeds



- Push the entire training and test data with features and targets down the neural network.
- First step: pre-processing and embeddings, which works differently for feature and target cells.

The encoder pre-processes and embeds

- Push the entire training and test data with features and targets down the neural network.
- First step: pre-processing and embeddings, which works differently for feature and target cells.

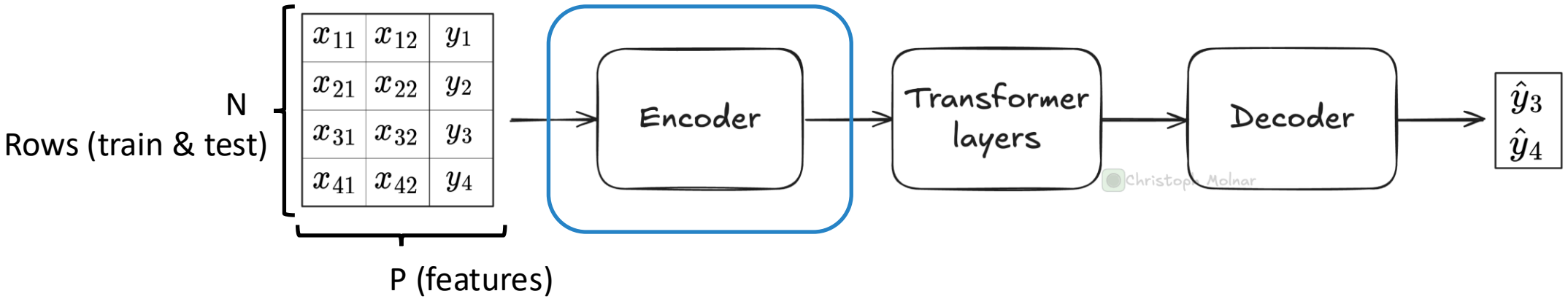
The feature encoder pre-processes and embeds feature cells:

- All features are normalized based on mean and std. dev. of training data.
- Values below -100 and above 100 are clipped.
- Features are linearly embedded.
  - Before embedding, cell's value is a scalar.
  - During embedding: a weight vector with length of embedding size E is multiplied with each scalar and a bias is added, producing E-length embedding vector for each cell. Weights are same for all feature cells

The target encoder does two things:

- The test valyes (y_test) are filled in with the mean of the target in the training data (mean(y_train)). Done for providing a starting value and not for imputation.
- The target values are linearly embedded, but with a weight vector different from the linear feature embedding.

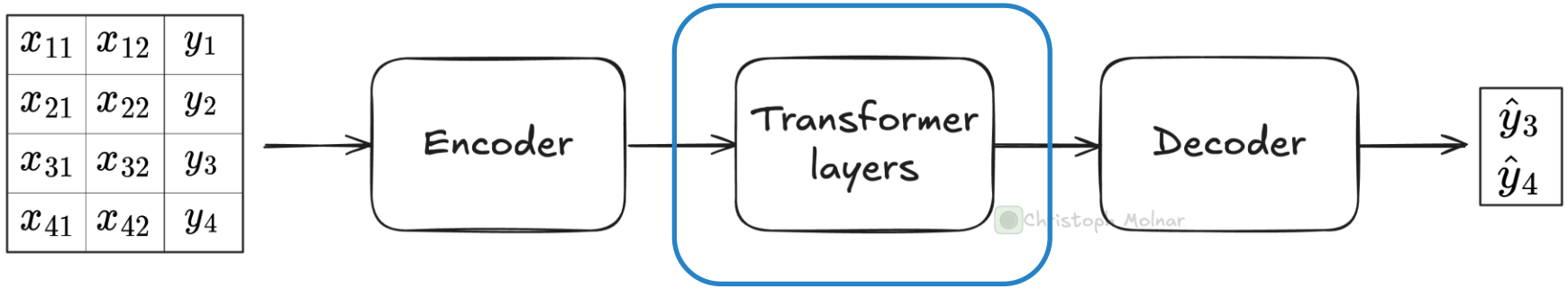
The encoder pre-processes and embeds



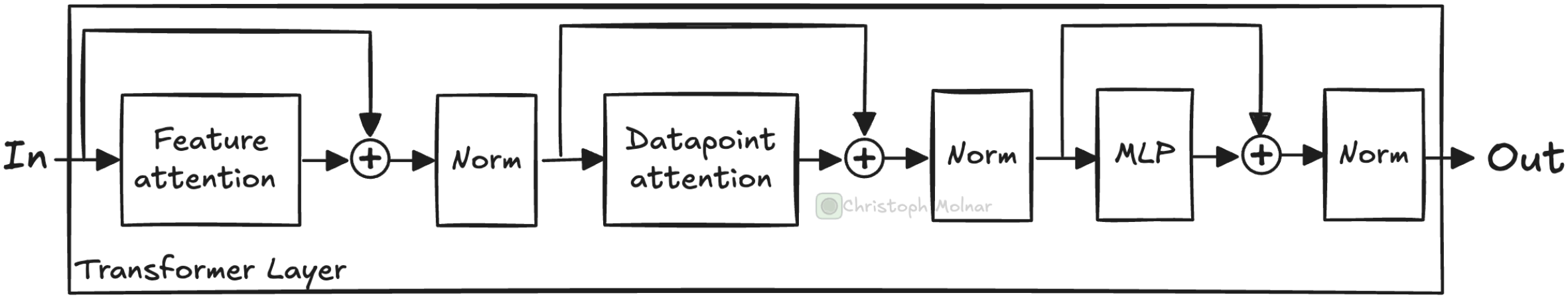
- Push the entire training and test data with features and targets down the neural network.
- First step: pre-processing and embeddings, which works differently for feature and target cells.
- nanoTabPFN has no positional embedding.
- TabPFN has a version of positional embedding for features (not the target).
  - After the linear embedding, the encoder adds a random vector to the cells of each feature.
  - This random vector is constructed via a projection, part learned, part random.
  - The positional embedding helps TabPFN better distinguish between the features.
- After encoding, cells no longer contain scalars, but vectors. The transformer layers operate on these vector representations.

The transformer layers route and aggregate information

The main computations happen within the transformer layers, which contain two attention layers: feature attention and datapoint attention.



The transformer layer route and aggregate information



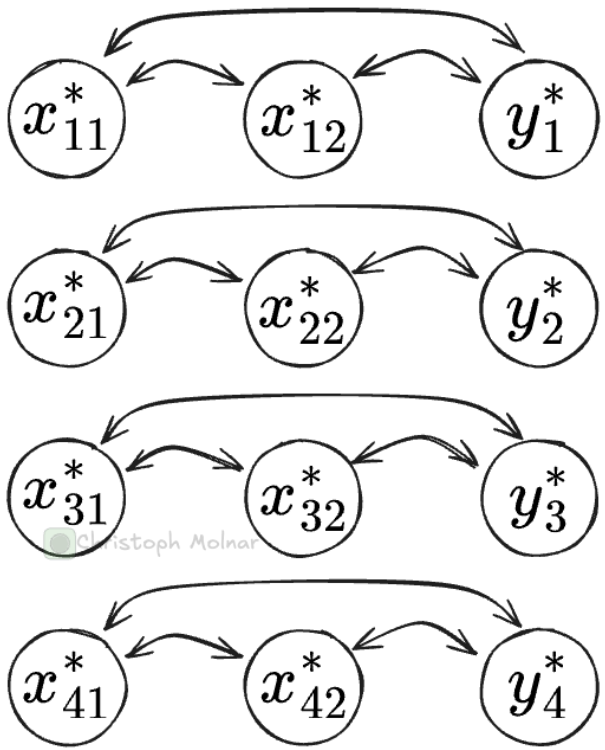
Schematic of a single transformer layer.

- TabPFN contain multiple transformer layers in sequence. A transformer layer in nanoTabPFN contains the following:
  - Feature attention: Attention mechanism that attends to cells in the same row.
  - Datapoint attention: Attention mechanism that attends to cells in the same column.
  - Multi-layer perceptron (MLP): A fully-connected network with 2 layers.
  - Layer norm: After each operation, the cell representations are normalized.
  - Skip connections, also called residual layers, before attention and MLP blocks.

The transformer layer route and aggregate information

Feature attention

- Allows attending to other cells in the same row, meaning from the same data point.
- After this attention step, each cell's vector representation is changed by the information contained in other cells of the same row (= same data point). In this step, the model can share information between all the features, including the target.

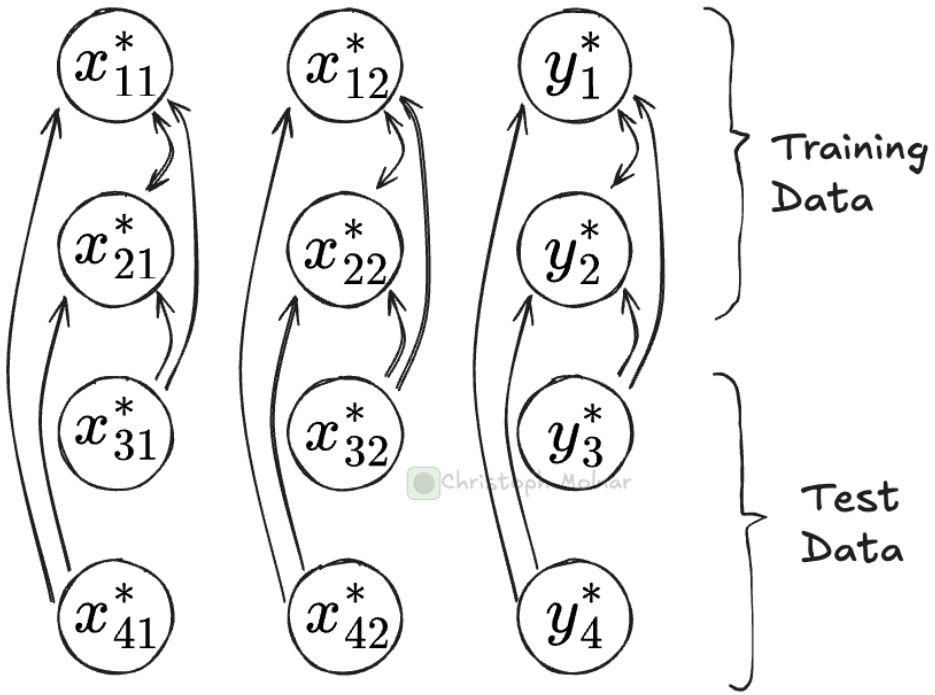


Feature attention. Note the asterisk on the cells: these are no longer the original cell values nor the initial embeddings (at least a bit deeper into the transformer). Instead, it's a vector of length 512 (for TabPFN) representing the cell.

The transformer layer route and aggregate information

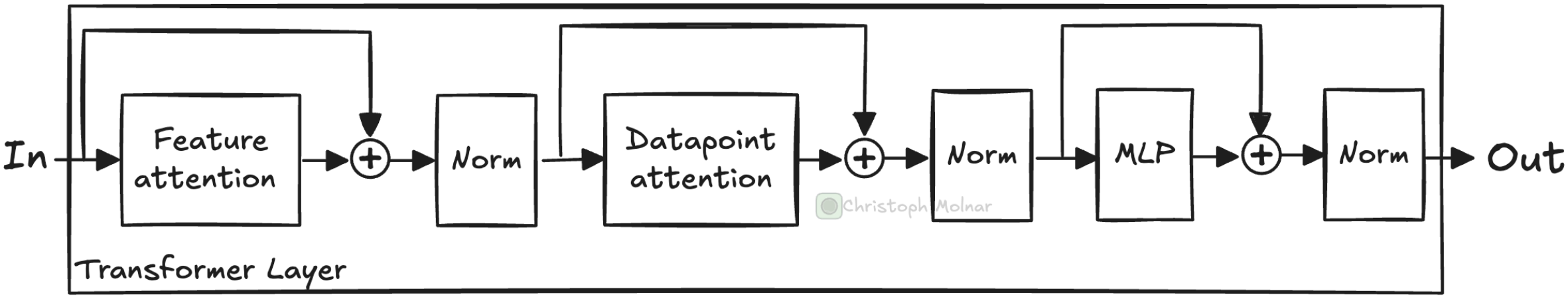
Datapoint attention

- Attends to cells from other data points.
- If you look at the vector representation for, say, cell x_12 (row 1, column 2), the datapoint attention layer can attend to other cells in the same column (and itself), for example, to x_22.
- This is somewhat similar to soft k-nearest neighbors: it finds the closest neighbor cells (from the same column) and pulls in some of their information.
- Datapoint attention distinguishes via masking whether a cell is from training or test data: cells can only attend to training data cells, but not to test data cells.



Datapoint attention. Cells may attend to other cells in the same column. However, they may only attend to cells from the training data, not from the test data.

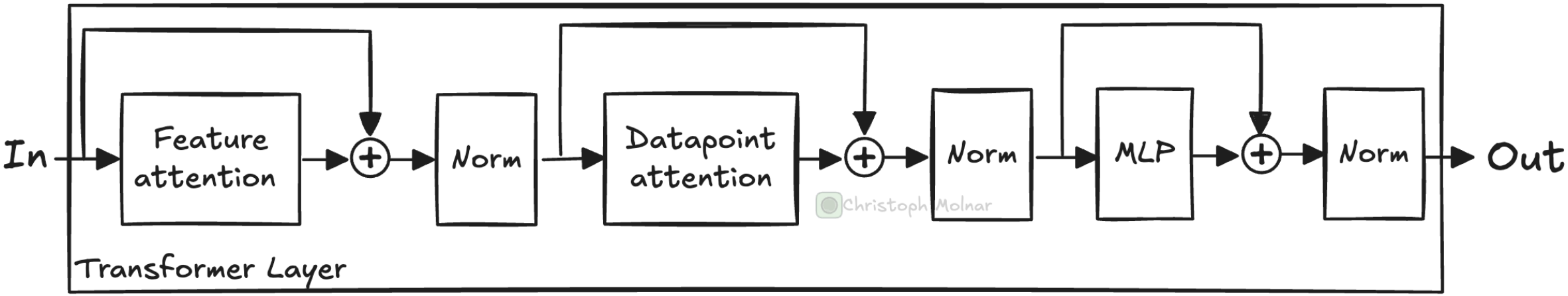
The transformer layer route and aggregate information



While the attention mechanisms are busy moving around information between cells, what is the **multi-layer perceptron (MLP)** doing?

- It consists of two fully-connected linear layers with a Gaussian Error Linear Unit (GELU) activation function in between.
- The MLP acts as the primary processing unit for the information from the attention mechanisms and also introduces the needed non-linearity to model more complex functions.
- While both attention mechanisms pull in information, the MLP takes in the embedding of a cell and can update the representation.

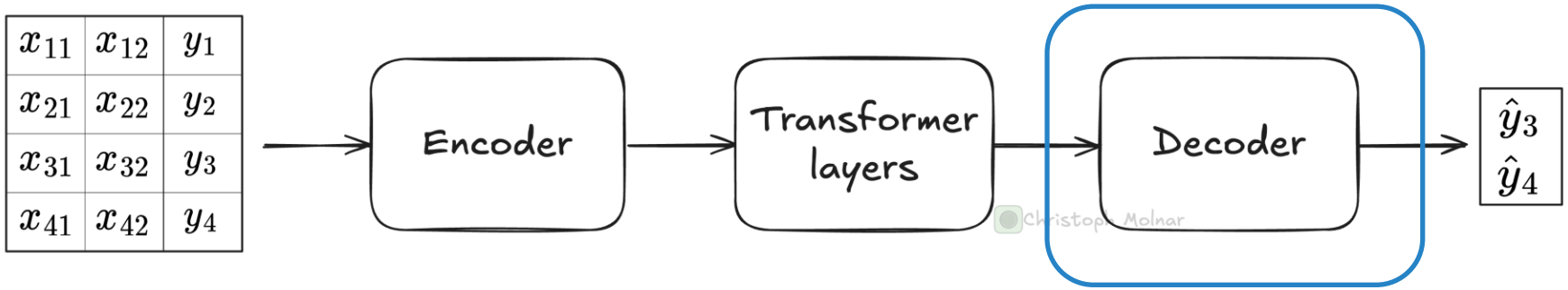
The transformer layer route and aggregate information



- The transformer layer is repeated, sequentially, a few times.
- Each time, the input to the next transformer is based on the output of the previous.
- Each iteration routes and aggregates information in the table.
- If TabPFN is anything like convolutional neural networks, perhaps with each transformer iteration, the model builds up more and more abstract representations of the tabular cells. TabPFN has 12 transformer layers, and nanoTabPFN has 3.

After the transformer layers, we have an enriched representation of our tabular dataset. In the next step, we decode this representation to obtain y_test.

The decoder predicts the test data



- The decoder takes as input all the embeddings of cells that represent the test targets (y_test), which, after the transformer layers, are now enriched representations containing information from other cells as well.
- These vectors are pushed through a MLP. Thereby, the same MLP is applied to each y_test cell individually to decode the representation into a prediction.
- For classification, the result is then pushed through a softmax layer, which for classification is fixed to 10 outputs.

Summary

- In short, during in-context learning for tabular models, the model treats the training data as a “dynamic lookup table”, routes information around the cells, and aggregates it, building up more and more abstract knowledge about the cell values.
- Note: the details presented here were about nanoTabPFN. TabPFN is more complex since it uses some tricks like feature grouping, adding random embeddings, and row hashing. Other TFMs, like TabICL, again have differences in their architectures.